

FieldProof

Work Verification & Automated
Invoicing Platform

FULL-STACK MERN APPLICATION

Presented by

Muhammad Hussain Memon

Course: Saylani Mass IT Training (Web & Mobile Development)

Project Type: Full-Stack MERN Web Application

MongoDB

Express.js

React.js

Node.js

SLIDE 01

FieldProof

Work Verification & Automated Invoicing Platform

Presenter: Muhammad Hussain Memon

Course: Saylani Mass IT Training (Web & Mobile Development)

Project Type: Full-Stack MERN Web Application

Detailed Introduction:

FieldProof is a smart web application built using the MERN stack (MongoDB, Express.js, React.js, and Node.js). It is designed to solve common time-tracking and attendance problems faced by companies with remote field workers.

In many service industries, managers cannot easily verify if a worker is actually present at the job site or if they are working the exact hours they claim. FieldProof solves this issue by combining GPS geo-fencing and mandatory photo proof. A worker can only check in when they are physically inside the allowed site area and upload a live photo.

Once the job is completed, the system automatically calculates the exact working hours and generates a clear bill (invoice) for the client. This removes manual paperwork, prevents fake attendance, and creates total trust between managers, field workers, and clients.

SLIDE 02

The Real World Problem

- **Fake Attendance:** Workers can lie about their working hours and exact location.
- **No Proof of Location:** Companies cannot verify if a worker was actually present on the job site.
- **Slow Billing Process:** Creating invoices manually takes days and often leads to human errors.
- **Lack of Photo Proof:** Clients cannot see photos or progress of completed work.

SLIDE 03

Our Solution

- **GPS Location Check:** Workers can only check in when they are inside the allowed job site area.
- **Photo Verification:** Workers must upload a photo when starting and finishing a task.
- **Automatic Invoices:** The system automatically calculates total hours and creates a bill.
- **Simple Dashboard:** Managers and clients can see live updates of all ongoing tasks.

SLIDE 04

Main Features

- **User Roles:** Separate access for Admins, Field Workers, and Clients.
- **GPS Location Lock:** Uses phone GPS coordinates to confirm real presence on site.
- **Photo Upload Engine:** Attaches time-stamped pictures directly to the specific task.
- **Auto Billing Engine:** Multiplies working hours with hourly rates to generate bills instantly.

SLIDE 05

Tech Stack (MERN)

Frontend (React & Tailwind CSS) Clean, mobile-friendly design that runs fast on all devices.	Backend (Node.js & Express.js) Fast server APIs to handle user requests, location checks, and logic.
Database (MongoDB & Mongoose) Stores user info, tasks, location logs, and generated invoices safely.	Security & Auth Uses JWT tokens and secure API cookies to protect account logins.

SLIDE 06

Project Directory & Folder Structure

Actual Backend MVC Architecture implementation:

```
backend/
├── node_modules/
├── src/
│   ├── config/
│   │   ├── database.js      # MongoDB Connection Config
│   │   └── env.config.js    # Environment Variables Config
│   ├── controllers/
│   │   ├── auth.controller.js # Auth Business Logic
│   │   └── task.controller.js # Task Management Logic
│   ├── middleware/
│   │   └── auth.middleware.js # JWT & Role Protection
│   ├── models/
│   │   ├── task.model.js    # Task & Geo-location Schema
│   │   └── user.model.js    # User Schema
│   ├── router/
│   │   ├── auth.route.js    # Auth API Endpoints
│   │   └── task.route.js    # Task Management Endpoints
│   └── services/
│       └── token.service.js  # JWT Token Generation Service
├── .env                    # Environment Variables
├── index.js                # Express Application Entry Point
├── package-lock.json
└── package.json
```

SLIDE 07

Backend API Endpoints

- **Authentication APIs (/api/auth)**
 - `POST /api/auth/register` — Create a new user account.
 - `POST /api/auth/login` — Login user and return JWT token.
- **Task APIs (/api/tasks)**
 - `POST /api/tasks/create` — Admin assigns a new task to a worker.
 - `GET /api/tasks/all` — Get list of all tasks.
 - `GET /api/tasks/task/:id` — Get specific task by ID.

SLIDE 08

Database Structure

- **Users:** Stores name, email, password hash, and role (Admin / Worker / Client).
- **Tasks:** Stores job title, site location, assigned worker, status, and deadline.
- **Logs:** Stores check-in time, check-out time, GPS coordinates, and uploaded photo link.
- **Invoices:** Stores bill amount, total hours worked, and payment status.

SLIDE 09

Core Backend Business Logic

Place VS Code code snippet screenshot here:

```
export const getTaskById = async (req, res) => {
  const { id } = req.params;
  const user = req.user;
  try {
    if(user.role !== 'admin' && user.role !== 'worker') {
      return res.status(403).json({ message: 'Access denied' });
    }
    const task = await Task.findById(id).populate('assignedWorker');
    if (!task) {
      return res.status(404).json({ message: 'Task not found' });
    }
    res.status(200).json( task );
  } catch (error) {
    res.status(500).json({ message: 'Error fetching task', error: error.message });
  }
}
```

SLIDE 10

Backend Testing — User Registration API

Endpoint: **POST** <http://localhost:4000/api/auth/register>

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** <http://localhost:4000/api/auth/register>
- Status:** 201 Created
- Size:** 586 Bytes
- Time:** 1.34 s

Request Body (JSON):

```
1 {
2   "name": "hussain",
3   "email": "hussain@gmail.com",
4   "password": "123456789"
5 }
```

Response Body (JSON):

```
1 {
2   "message": "User registered successfully",
3   "newUser": {
4     "name": "hussain",
5     "email": "hussain@gmail.com",
6     "password":
7       "$2b$10$0oWgJzVKJElbr1GdUMB1m0U4E2RXjoo44F
8       IFm/wmmdcJUXGricPSK",
9     "role": "worker",
10    "phone": "",
11    "hourlyRate": 0,
12    "avatar": "",
13    "id": "6aab5d182c7996764b1acad3",
14    "createdAt": "2026-09-17T03:23:04.041Z",
15    "updatedAt": "2026-09-17T03:23:04.041Z",
16    "__v": 0
17  },
18   "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
19   .eyJpZCI6IjZhYWI1ZDE4MmM3OTk2NzY0YjFhY2FkMyI
20   sInJvbGUioiIj3b3JrZXIiLCJlbWFPbC1mhaW5
21   AZ21hakWuY29tIiwiaWF0IjoxNzE1Mzg0LCJleHA"
22 }
```

SLIDE 11

Backend Testing — Create Task API

Endpoint: **POST** <http://localhost:4000/api/tasks/create>

The screenshot shows a REST client interface with the following details:

- Method:** POST
- URL:** <http://localhost:4000/api/tasks/create>
- Status:** 201 Created
- Size:** 524 Bytes
- Time:** 515 ms

Request Body (JSON):

```
1 {
2   "title": "AC Maintenance & Repair",
3   "description": "Servicing and gas refilling for
4   main server room AC units.",
5   "assignedWorker": "6aab5d182c7996764b1acad3",
6   "dueDate": "2026-09-25T18:00:00.000Z",
7   "siteLocation": {
8     "name": "Zaitoon Ashraf IT Park",
9     "latitude": 24.8607,
10    "longitude": 67.0011,
11    "radiusInMeters": 100
12  }
13 }
```

Response Body (JSON):

```
1 {
2   "message": "Task created successfully",
3   "task": {
4     "title": "AC Maintenance & Repair",
5     "description": "Servicing and gas refilling for
6     main server room AC units.",
7     "assignedWorker": "6aab5d182c7996764b1acad3",
8     "createdBy": "6aab5d182c7996764b1acad3",
9     "siteLocation": {
10      "name": "Zaitoon Ashraf IT Park",
11      "latitude": 24.8607,
12      "longitude": 67.0011,
13      "radiusInMeters": 100
14    },
15     "status": "pending",
16     "dueDate": "2026-09-25T18:00:00.000Z",
17     "id": "6aab708ee300c8ed264b784d",
18     "createdAt": "2026-09-17T04:46:06.963Z",
19     "updatedAt": "2026-09-17T04:46:06.963Z",
20     "__v": 0
21   }
22 }
```

SLIDE 12

Backend Testing — Get All Tasks API

Endpoint: GET `http://localhost:4000/api/tasks/all`

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `http://localhost:4000/api/tasks/all`
- Status:** 200 OK
- Size:** 1.32 KB
- Time:** 871 ms
- Response Body (JSON):**

```
{  "siteLocation": {    "name": "Main Office Building - Floor 3",    "latitude": 24.8607,    "longitude": 67.0011,    "radiusInMeters": 100  },  "id": "6aab6b2a5ecc4191c490fc3d",  "title": "AC Maintenance & Repair",  "description": "Servicing and gas refilling for main server room AC units.",  "assignedWorker": {    "_id": "6aab5f139c1170f38824c7ce",    "name": "hussain",    "email": "hussain123@gmail.com",    "role": "worker",    "phone": "",    "hourlyRate": 200,    "avatar": "",    "createdAt": "2026-09-17T03:31:31.871Z",    "updatedAt": "2026-09-17T03:31:31.871Z",    "__v": 0  },  "createdBy": "6aab5d182c7996764b1ac4d3",  "createdBy": "6aab5d182c7996764b1ac4d3",  "status": "pending"}
```

SLIDE 13

Backend Testing — Get Task By ID API

Endpoint: GET `http://localhost:4000/api/tasks/task/:id`

The screenshot shows a REST client interface with the following details:

- Method:** GET
- URL:** `http://localhost:4000/api/tasks/task/6aab6b2a5ecc4191c490fc3d`
- Status:** 200 OK
- Size:** 683 Bytes
- Time:** 693 ms
- Response Body (JSON):**

```
{  "siteLocation": {    "name": "Main Office Building - Floor 3",    "latitude": 24.8607,    "longitude": 67.0011,    "radiusInMeters": 100  },  "id": "6aab6b2a5ecc4191c490fc3d",  "title": "AC Maintenance & Repair",  "description": "Servicing and gas refilling for main server room AC units.",  "assignedWorker": {    "_id": "6aab5f139c1170f38824c7ce",    "name": "hussain",    "email": "hussain123@gmail.com",    "role": "worker",    "phone": "",    "hourlyRate": 200,    "avatar": "",    "createdAt": "2026-09-17T03:31:31.871Z",    "updatedAt": "2026-09-17T03:31:31.871Z",    "__v": 0  },  "createdBy": "6aab5d182c7996764b1ac4d3",  "status": "pending"}
```

Thank You!

Questions & Feedback

Presented by

Muhammad Hussain Memon

Saylani Mass IT Training • Web & Mobile Development

MongoDB

Express.js

React.js

Node.js